

CS471: Introduction to Artificial Intelligence

Assignment 4: Decision Tree Classification

Shane Dyrdaahl

10/24/24

In this assignment, you will implement the Decision tree classification method using **Scikit-learn**.

This is an individual assignment. Use Google Colab for your code, plots, and comments. When you finish editing, re-run all the cells to make sure they work.

In this assignment, you will work with the Iris dataset that was used in [R. A Fisher's paper](#). You can also find the dataset in the [UCI Machine Learning Repository](#). You can directly load the dataset using sklearn:

```
from sklearn.datasets import load_iris
data = load_iris()
```

Tasks:

1. Include a basic description of the data (what are the features and labels) (1 point)
Write in your own words of what the classification task is and why a decision tree is a reasonable model to try for this data. (1 point)
2. Split the data into training, validation, and testing sets. (1 point)
3. Fit a decision tree on the training dataset. (1 point)
4. Tune at least 2 hyperparameters in the decision tree model
(<https://ken-hoffman.medium.com/decision-tree-hyperparameters-explained-49158ee1268e>)
based on the performance on the validation set or using cross-validation. One hyperparameter has to be max_depth and the other one is your choice. Generate plot of hyperparameter values vs performance metric (plots are mandatory). (4 points)
5. Train the model using optimal hyperparameters (found in step 4) on the train + validation data. Test it on test data and generate a classification report (1 point)
6. Inspect the model by visualizing and interpreting the results (1 point)

GoogleCollab:

<https://colab.research.google.com/drive/1EVhXFPPAFZGtmTFggncNTkkAhcj51uqO?usp=sharing>

1. The Iris dataset consists of 150 samples from three species of Iris flowers: Setosa, Versicolor, and Virginica. It is characterized by four numeric features: sepal length, sepal width, petal length, and petal width. These features are important for predicting the species of the Iris flower. The dataset is balanced, with 50 samples for each species, ensuring reliable results. The target label (species) is categorical, being one of the following: Setosa, Versicolor, or Virginica.

The classification task involves predicting the species of an Iris flower based on the four given features. This is a supervised learning task, where we use labeled data to train a model that can classify new, unseen samples.

A decision tree is a fitting model for this task due to its ability to handle numeric features and categorical labels. More importantly, decision trees are highly interpretable, providing a visual representation-making process. The Iris dataset, being relatively small and straightforward, is well-suited for a decision tree model, which efficiently captures the relationships between the features and labels without overcomplicating the model.

2. I load the dataset from scikit-learn with `load_iris()`. I then extract the features and labels from the Iris dataset.

```
iris = load_iris()
X, y = iris.data, iris.target
```

I then split the data into 30% testing and 60% training sets. The `random_state` makes it so that the split is reproducible across different runs.

```
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.4, random_state=42)
```

3. I create a decision tree, passing `random_state = 3` as a parameter. This ensures that I can control the randomness so that I can have reproducible results. This helps with all the parameters that can be adjusted by me, like how we split up the dataset.

```
# 3) Fit a decision tree on the training dataset.
clf = DecisionTreeClassifier(random_state=3)
clf.fit(X_train, y_train)
```

4.

Tuning:

I define the chosen hyperparameters 'max_depth' and 'min_samples_split' to a grid.

```
# Hyperparameter to fine tune
param_grid = {
    'max_depth': range(2, 20, 1),
    'min_samples_split': range(2, 20, 1)
}
```

I use **GridSearchCV()** to perform a grid search with cross-validation.

The parameters that are passed are:

- estimator=clf $\Rightarrow$ this is how I chose to use the decision tree classifier that I created in step 3.
- param_grid=param_grid $\Rightarrow$ choosing to use the hyperparameter grid
- cv=5 $\Rightarrow$ this is choosing to use 5 folds in the cross validation
- scoring='accuracy' $\Rightarrow$ This tells the grid search to use accuracy as the evaluation metric when determining how well each hyperparameter combination performs.

The **fit(X_train, y_train)** method in GridSearchCV runs the grid search by training and evaluating the model. It selects the best hyperparameters based on accuracy, providing an optimized model.

```
grid_search = GridSearchCV(estimator=clf, param_grid=param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
```

Plotting:

The **grid_search.cv_results_** stores detailed results from the grid search, including mean test scores for each combination of hyperparameters.

I aggregate the mean cross-validation accuracy for each max_depth

```
max_depths = np.array(results['param_max_depth'].data, dtype=int)
mean_scores = np.array(results['mean_test_score'])
```

I get unique max_depth values and then I calculate the mean accuracy for each unique max_depth.

```
unique_depths = np.unique(max_depths)
mean_accuracy_per_depth = [
    np.mean(mean_scores[max_depths == depth]) for depth in unique_depths
]
```

I do the same thing for the min_samples_split

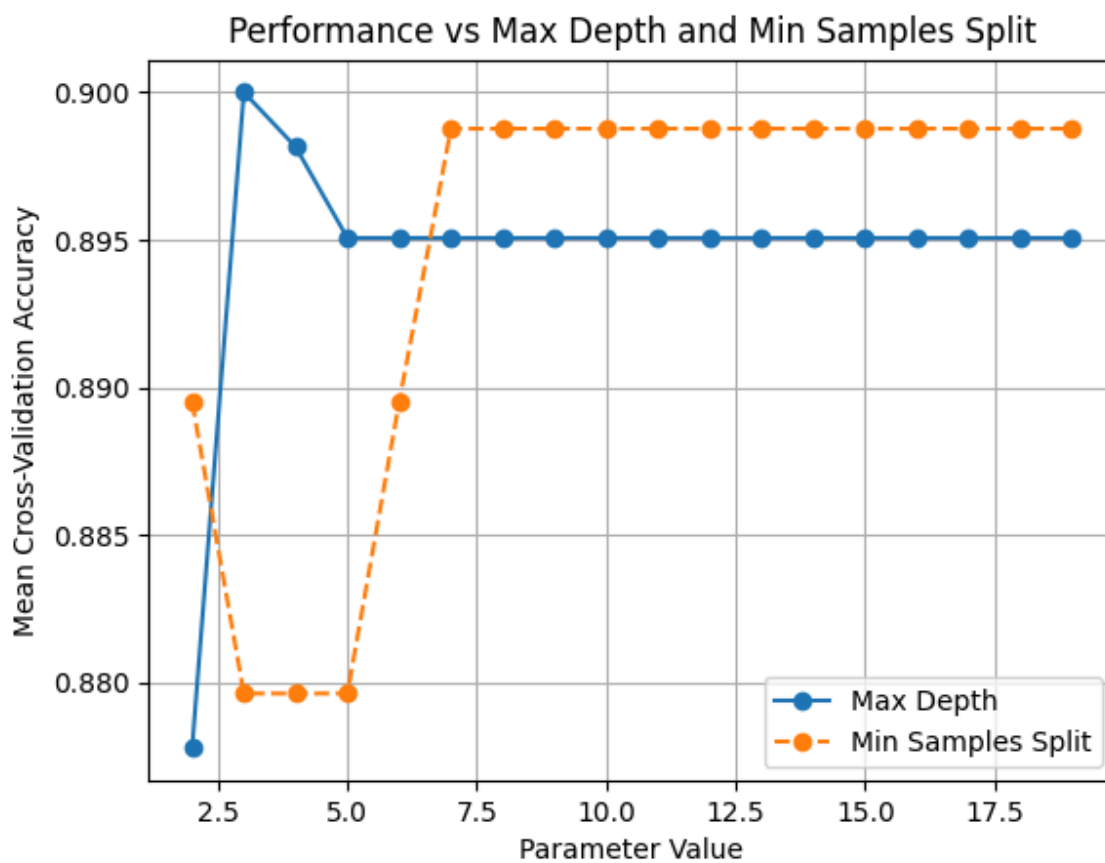
```
# Aggregate the mean cross-validation accuracy for each min_samples_split
min_samples_splits = np.array(results['param_min_samples_split'].data, dtype=int)

# Get unique min_samples_split values
unique_splits = np.unique(min_samples_splits)

# Calculate the mean accuracy for each unique min_samples_split
mean_accuracy_per_split = [
    np.mean(mean_scores[min_samples_splits == split]) for split in unique_splits
]
```

I then plot these two onto the same graph

```
# Plot max_depth & min_samples_split vs mean cross-validation accuracy
plt.plot(*args: unique_depths, mean_accuracy_per_depth, marker='o', label='Max Depth')
plt.plot(*args: unique_splits, mean_accuracy_per_split, marker='o', label='Min Samples Split', linestyle='--')
plt.xlabel('Parameter Value')
plt.ylabel('Mean Cross-Validation Accuracy')
plt.title('Performance vs Max Depth and Min Samples Split')
plt.legend(['Max Depth', 'Min Samples Split'])
plt.grid(True)
plt.show()
```



5. After tuning, the best model (best_estimator) is used to make predictions on both the training (X_train) and test sets (X_test). I then print out a classification report.

```
y_train_pred = best_estimator.predict(X_train)
final_train_accuracy = accuracy_score(y_train, y_train_pred)
print('\nTraining accuracy: {}'.format(final_train_accuracy))

y_test_pred = best_estimator.predict(X_test)
final_test_accuracy = accuracy_score(y_test, y_test_pred)
print('Testing accuracy: {}'.format(final_test_accuracy))

# Generate a classification report
print("\nClassification Report:\n", classification_report(y_test, y_test_pred))
```

6. I create a decision tree using plot_tree() in order to better visualize the data.

```
plt.figure(figsize=(20, 10))
plot_tree(best_estimator, filled=True, feature_names=iris.feature_names, class_names=iris.target_names, rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```

We can see from this tree that it successfully classifies each species, with the leaf nodes representing accurate or nearly accurate classifications for all samples. This decision tree classifies iris flowers based on petal width and petal length, with the first split at petal width ≤ 0.8 perfectly separating all Setosa samples. The remaining samples (Versicolor and Virginica) are further classified using petal width and petal length. Versicolor is primarily separated at petal length ≤ 5.35 , and Virginica at petal length > 4.85 .

